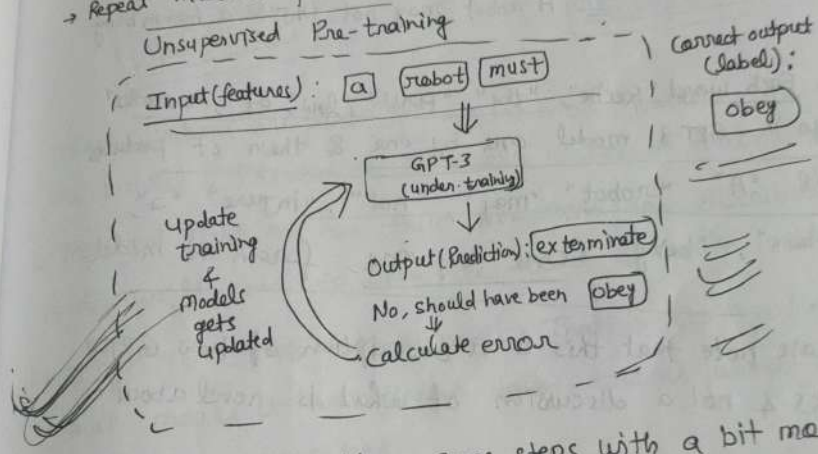
The model is presented with an example. We only show it the features & ask it to predict the next word.

The model's prediction will be wrong. We calculate the error in its prediction & update the model so next time it makes a better prediction.

→ Repeat millions of times.



↳ Now, let's look at these same steps with a bit more detail.

GPT3 actually generates output one token at a time (let's assume a token is a word for now).

→ Input Prompt: [Recite] [the] [first] [law] [of] [robotics]

↓

GPT-3

↓

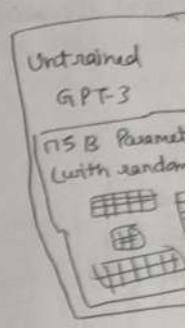
o/p: A robot may not injure a human being

↳ Each word "Recite", "the", "first", "law", "of", "robotics" go in GPT-3 model one by one & then it produces o/p "A", "robot", "may", "not", "injure", "a", "human", "being" one by one from the model.

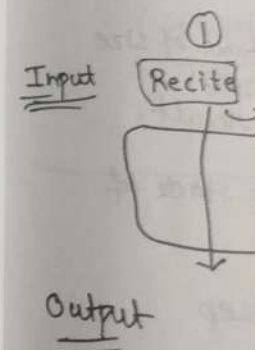
↳ Please note that this is a description of how GPT-3 works & not a discussion of what is novel about it.

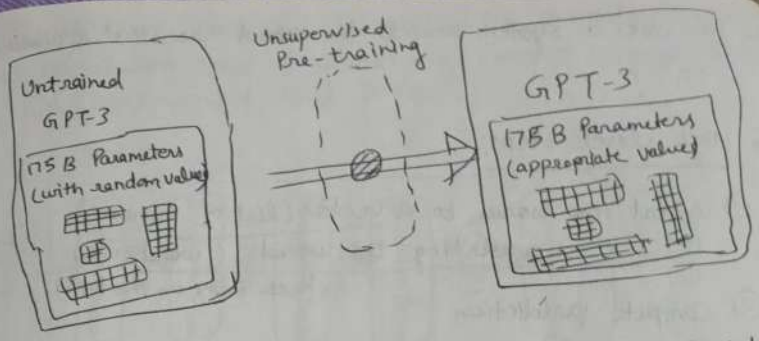
↳ GPT-3 is MASSIVE. It encodes what it learns from training in 175 billion numbers (called parameters). These numbers are used to calculate which token to generate at each run.

The untrained model starts with random parameters. Training finds values that lead to better prediction.



These number
the model.
↳ To shed lig
used, we'll
↳ GPT-3 is
That mea
tokens a

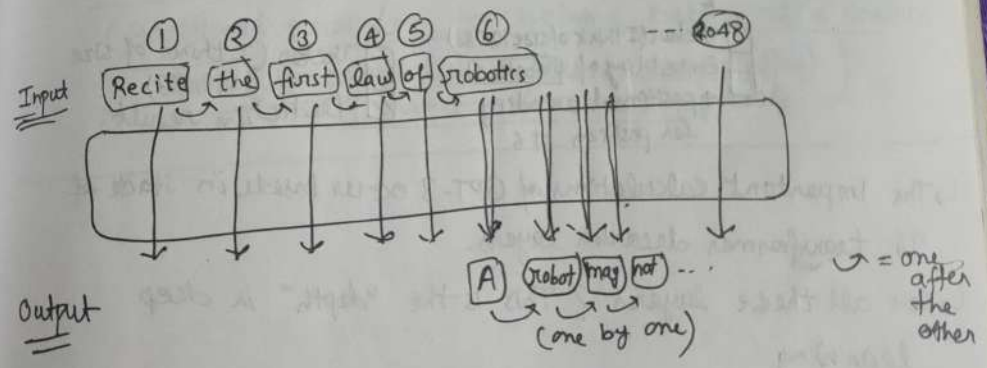




These numbers are part of hundreds of matrices inside the model. Prediction is mostly a lot of matrix mul".

↳ To shed light on how these parameters are distributed & used, we'll need to open the model & look inside.

↳ GPT-3 is 2048 tokens wide. That is its "context window". That means it has 2048 ~~tok~~ tracks along which tokens are processed.



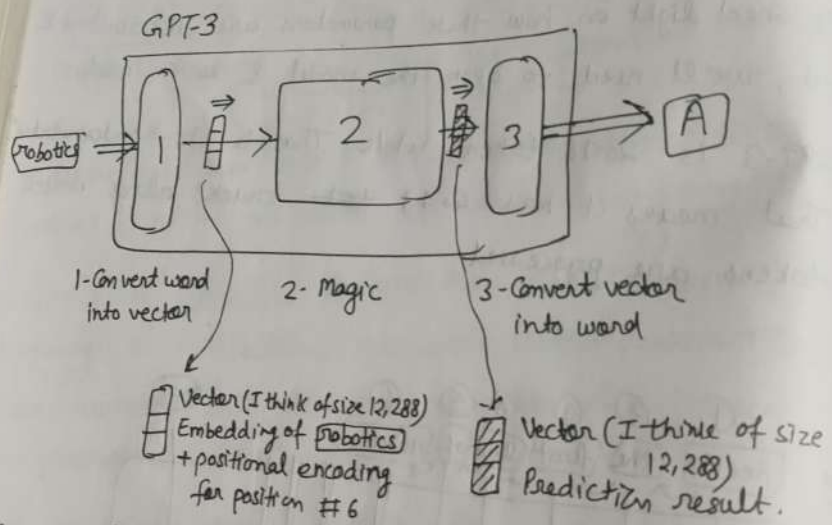
↳ How does a system process the word "robotics" & produce "A"?

↳ High-level steps:

① Convert the word to a vector (list of numbers) representing the word (word2vec) (check blog on this topic)

② Compute prediction

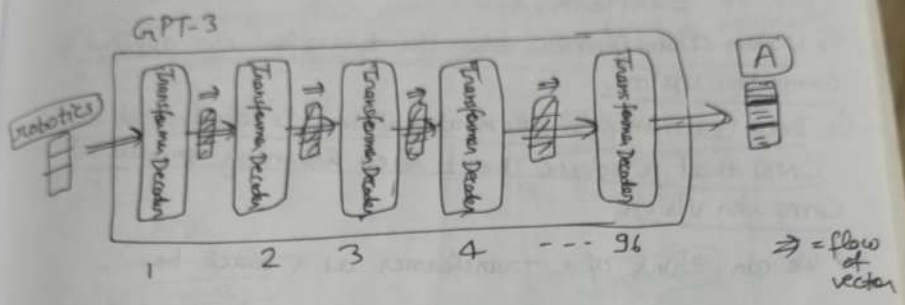
③ Convert resulting vector to word



↳ The important calculations of GPT-3 occur inside its stack of 96 transformer decoder layers.

↳ See all these layers? This is the "depth" in deep learning.

↳ Each of these layers has its own 1.8 B parameter to make its calculations. This is where the "magic" happens. This is a high-level view of that process:



↳ You can see a detailed explanation of everything inside the decoder in blog post [The Illustrated GPT-2]

↳ The difference with GPT3 is the alternating dense & sparse self-attention layers.

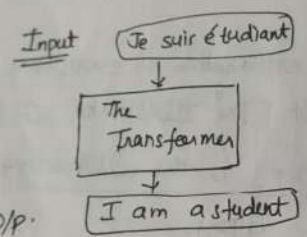
⇒ Fine-tuning actually updates the model's weights to make the model better at a certain task.
(Additional training to become better at a certain task [for ex. English to french translation])
(parameter values gets updated)

→ Transformer basically is a type of ML model or it's an architecture of NN. Various variants of transformers are there like BERT that builds on the top of transformers.

↳ Vision Transformers are the transformers applied to computer vision.

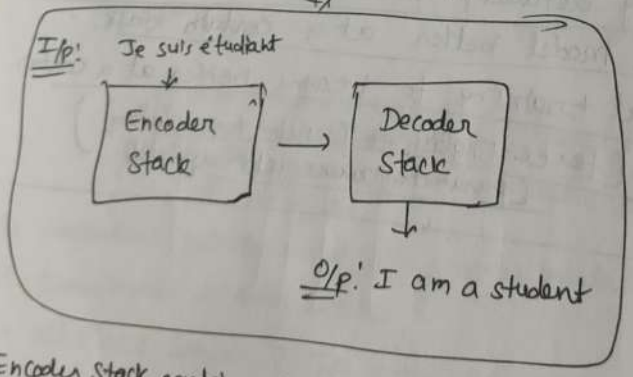
↳ Deep learning in computer vision is dominated by CNN that produce best deep learning models in computer vision.

⇒ We can think of a transformer as a black box.



This is from the original paper bcoz paper talked about the machine Translation initially.

⇓ can be explored as



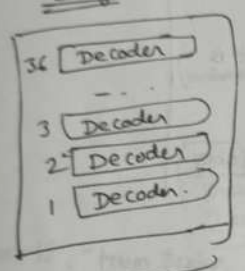
↳ Encoder Stack contains 6 encoder block & Decoder stack contains 6 decoder block in initial transformers. [These are building blocks of transformers]

→ Transformer is Encoder-Decoder based model but Transformer Language model is decoder based model. (Ex. GPT-2, GPT-3 etc).

→ ~~Transformers which are based on Encod~~

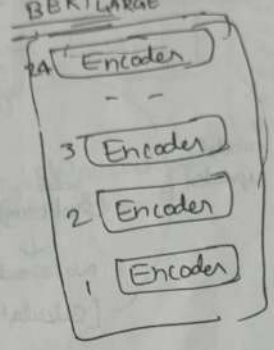
→ Transformers based Encoder models has example BERT etc.

⇒ GPT-2 Large



36 blocks of Decoders

BERT LARGE

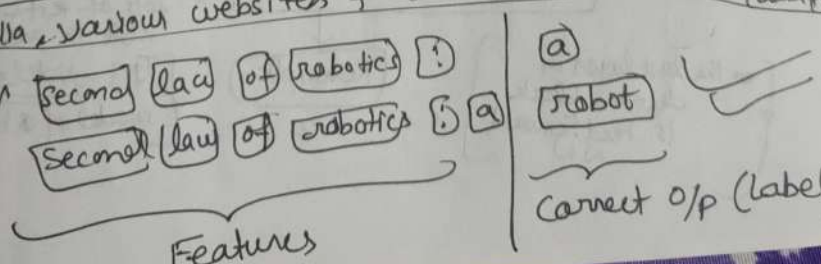


24 blocks of Encoders

↳ A model starts as untrained & it goes through a pre-training process where we expose it to a lot of languages & from that emerges a trained language & in this case, we have a lot of data that we train the model. For language models, data is text, we get lots of data from internet, wikipedia, various websites & we create training examples. (data points)

ex.

dataset



The above training example is generated from the text

Text: "Second law of Robotics: A robot must obey the orders given it by human beings."

↳ Once we have our examples, we present them like

I/p (features): [a] [robot] [must]

↓

GPT-3 (under training) (Model is not trained)

↓

O/p: [troll] (It is random)
(Prediction)

⇓

No, should have been [obey]

[Calculate error]

model updated

Correct O/p (labels)

[obey]

Now, next time when model sees "a robot must", it more likely to give "obey". We do this millions of times & train the model.

[This is the pre-training phase of a language model]

⇒ Now,

Input: 0 [The] 1 [Shashwank]

↓ ↓

Transformer (Language Model) [Decoder] ~~(Feed~~

↓

[Redemption]

2

The model has looked through like wikipedia & has been trained on lots of text & so it will able to predict.

[Try: distilgpt2 model of Hugging Face]

[the last layer in decoder block is Feed Forward NN]

↳ The 2 main

② Feed Fo

⇒ Tokenizatio

Input:

Tokenize:

Convert to token-ids

↳ The 2 main components of Decoder block is ① Self Attention ② Feed Forward NN.

→ Tokenization

Input: The Shawshank

Tokenize: 'The' 'shaw' 'sh' 'ank'

0	1	2	3
464	18193	1477	962

Convert to token_ids

These integers represent the index of that token in the vacabulary

This is presented to the model

Transformer (Language Model)

34433 → Token_Id

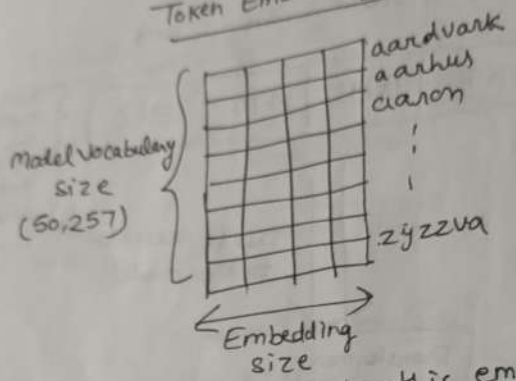
Tokenizer translate this token id to a word.

Redemption

⇒ Breathing Numbers into meaning into Numbers.

↳ When we downloaded a pre-trained GPT-2 model it came with this list:

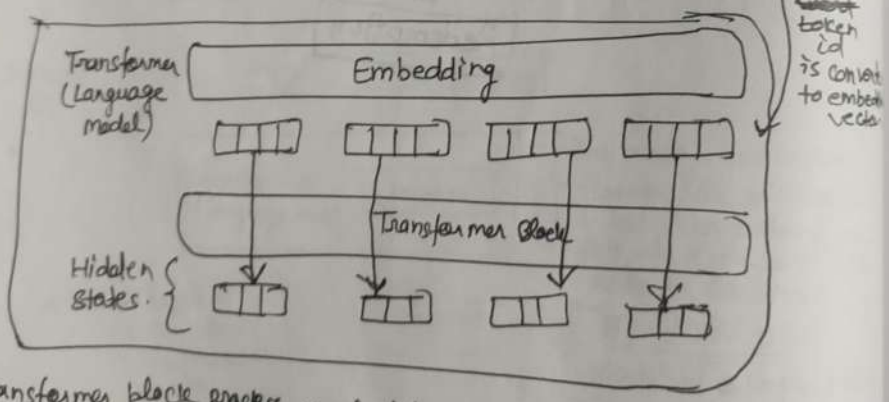
Token Embeddings (wte)



↳ Part of training is to create this embedding matrix

So,

'The'	'Shaw'	'Sh'	'ank'
464	18193	1477	962



↳ Transformer block process each token at the same time parallely. & output a vector & then next block starts..

↳ To capture re
one block,
& so on.

↳ For language
on the m

⇒ Turning

Project
to voca

Logits: A scor
each token
vocabula